



MODUL 8 · STATE MANAGEMENT

Prinsip Server-First State di Next.js 16

Berkenalan dengan state yang ada di Server maupun Client



Pertanyaan Pertama: Apakah Ini Benar-Benar Client State?

Kebanyakan developer terlalu cepat reach for useState atau Zustand. Cek checklist ini dulu.



Apakah data ini dari database/API?

→ Server Component. Fetch langsung di sana.



Apakah state ini perlu di-share antar halaman?

→ Mungkin URL (searchParams). Bisa di-bookmark, shareable.



Apakah ini form input sebelum submit?

→ useState lokal. Tidak perlu global state library.



Apakah ini perlu persist di localStorage?

→ Zustand dengan persist middleware. Tapi apakah perlu?



Apakah data berubah setelah mutasi user?

→ Server Action + revalidateTag + router.refresh(). Bukan useState.

URL Sebagai State — Underrated Pattern

Search, filter, sort, pagination — semuanya lebih baik di URL daripada di useState.

✗ State lokal — tidak shareable

```
'use client'
import { useState } from 'react'

function BlogPage() {
  // State hilang saat refresh!
  // Tidak bisa di-share link-nya.
  const [search, setSearch] = useState('')
  const [category, setCategory] = useState('all')
  const [page, setPage] = useState(1)

  const filtered = posts
    .filter((p) => p.title.includes(search))
    .filter((p) => category === 'all' || p.category ===
category)
  // "Eh, kamu filter apa tadi?"
  // → Tidak bisa kirim link yang sama!
  return <PostList posts={filtered} />
}
```

✓ URL state — shareable & bookmarkable

```
// Server Component — tidak butuh "use client"!
// URL: /blog?q=nextjs&cat=tutorial&page=2
export default async function BlogPage({
  searchParams,
}): {
  searchParams: Promise<{
    q?: string,
    cat?: string,
    page?: string,
  }>,
} {
  const { q = '', cat = 'all', page = '1' } = await
searchParams
  // Fetch berdasarkan URL params
  const posts = await getPosts({
    query: q,
    category: cat,
    page: parseInt(page),
  })

  return (
    <div>
      <SearchInput defaultValue={q} />
      <PostList posts={posts} />
      <Pagination page={parseInt(page)} />
    </div>
  )
}
```

useOptimistic — Optimistic UI Tanpa State Library

React 19: built-in hook untuk optimistic updates. Menggantikan banyak pola Zustand lama.

useOptimistic — like button yang responsive

```
"use client"
import { useOptimistic, useTransition } from "react"
import { toggleLike } from "@actions/post"

interface LikeButtonProps {
  postId: string
  initialLiked: boolean
  likeCount: number
}

export function LikeButton({
  postId, initialLiked, likeCount,
}: LikeButtonProps) {
  const [isPending, startTransition] = useTransition()
  const [optimisticLiked, setOptimisticLiked] = useOptimistic(initialLiked)

  function handleLike() {
    startTransition(async () => {
      setOptimisticLiked(!optimisticLiked) // ← UI update LANGSUNG
      await toggleLike(postId)              // ← Server Action (async)
      // Jika server error → otomatis rollback ke initialLiked!
    })
  }

  return (
    <button onClick={handleLike} disabled={isPending}>
      {optimisticLiked ? "❤️" : "♥️"}
      {likeCount + (optimisticLiked !== initialLiked ? (optimisticLiked ? 1 : -1) :
0)} </button>
    )
  )
}
```

Kapan Benar-Benar Butuh Client State Library?

Setelah mengeliminasi server state, URL state, dan local state — ini yang tersisa.

Tidak perlu library

- Data dari DB → Server Component
- Filter/sort/pagination → URL searchParams
- Form input → useState lokal
- Optimistic update → useOptimistic
- Data setelah mutasi → revalidateTag + refresh
- Simple toggle (modal, sidebar) → useState lokal

Butuh Context atau Zustand

- Theme (dark/light) — shared across tree
- Shopping cart — persist, accessible mana saja
- User preferences — cross-component, non-server
- Multi-step form state — antar step
- Complex filter UI state > 5 filter terkait
- Real-time notification state (WebSocket)

Live Coding — Refactor: useState ke URL State

Ganti client state search/filter ke URL searchParams. Lebih baik di semua dimensi.

01

Identifikasi

Di `/blog/page.tsx`, cari semua `useState` yang menyimpan filter, search, category, page. List semua yang bisa dipindah ke URL.

02

Server Component

Ubah `BlogPage` menjadi `async Server Component`. Baca filter dari `searchParams` (`await`). Fetch data dengan filter tersebut.

03

SearchInput Client

Buat `SearchInput` 'use client' yang hanya terima `defaultValue` dan update URL dengan `useRouter().push()` ketika user mengetik.

04

useOptimistic

Tambahkan `LikeButton` menggunakan `useOptimistic` + `toggleLike` Server Action. Tanpa `useState`, tanpa library — just React 19.

Setelah refactor: coba copy-paste URL filter ke tab baru → hasil sama. Coba browser Back → filter kembali ke sebelumnya. Ini gratis dari URL state.

Evaluasi Pemahaman

Jawab sebelum lanjut ke slide berikutnya.

Q1 — User mengklik filter 'Next.js' di halaman blog. State mana yang paling tepat?

- A) useState lokal di Client Component
- B) URL searchParams (?category=nextjs) — shareable, bookmarkable
- C) Zustand store global

Q2 — Apa keuntungan useOptimistic dibanding useState untuk like button?

- A) Lebih cepat karena tidak butuh render
- B) UI update langsung, auto-rollback jika server error — tanpa manual state management
- C) Bisa dipakai di Server Component

Q3 — Kapan barulah kamu benar-benar butuh Zustand atau Context?

- A) Untuk setiap state yang lebih dari satu komponen
- B) Setelah eliminasi server state, URL state, dan local state — untuk UI state yang genuinely shared
- C) Selalu — lebih aman pakai Zustand dari awal

Tugas Mandiri

Audit project kamu: berapa banyak useState yang bisa digantikan pola lebih baik?

01 State Audit

List semua useState di project. Untuk setiap satu: apakah bisa server state? URL state? atau memang perlu client state?

02 URL Refactor

Pilih satu halaman dengan filter/search yang pakai useState. Refactor ke URL searchParams. Pastikan back/forward browser bekerja.

03 useOptimistic

Implementasikan LikeButton menggunakan useOptimistic + Server Action. Test: klik like → UI update langsung. Simulasikan error → verify rollback.

04 Decision Doc

Buat doc pendek: state mana di project yang butuh Context? Yang butuh Zustand? Yang bisa dihilangkan? Bawa ke sesi Bab 2.



Goal: kurangi jumlah useState sebanyak mungkin sebelum perlu reach for Context atau Zustand.

Yang Sudah Kita Pelajari

- ✓ Server-first: data dari DB → Server Component. Bukan useState, bukan Zustand.
- ✓ URL state untuk filter/sort/pagination — shareable, bookmarkable, tanpa library.
- ✓ useOptimistic (React 19): optimistic UI dengan auto-rollback. Menggantikan banyak pola Zustand lama.
- ✓ State hierarchy: Server → URL → local useState → Context → Zustand. Climb hanya jika perlu.
- ✓ Zustand hanya untuk UI state yang genuinely shared dan tidak bisa di-URL atau di-server.

→ Selanjutnya: **Bab 2 — React Context: Kapan Sudah Cukup**

